

Conditions.

IF Syntax.

IF Condition THEN { StatementBlock
NEXT SENTENCE }
ELSE { StatementBlock
NEXT SENTENCE } [END - IF]

CONDITION TYPES

Simple Conditions

Relation Conditions

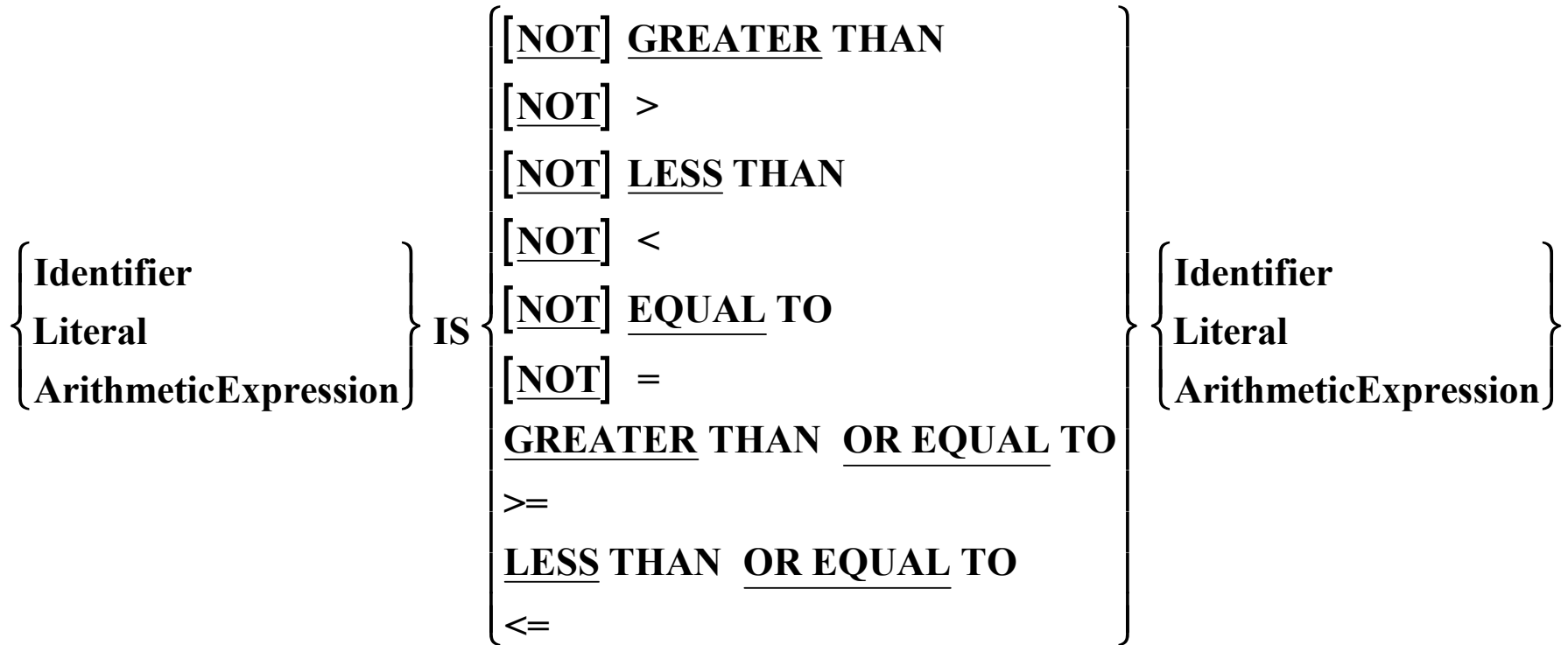
Class Conditions

Sign Conditions

Complex Conditions

Condition Names

Relation Conditions



Class Conditions.

Identifier IS [NOT] {
NUMERIC
ALPHABETIC
ALPHABETIC - LOWER
ALPHABETIC - UPPER
UserDefinedClassName

Although COBOL data items are not 'typed' they do fall into some broad categories, or classes, such a numeric or alphanumeric, etc.

A Class Condition determines whether the value of data item is a member of one these classes.

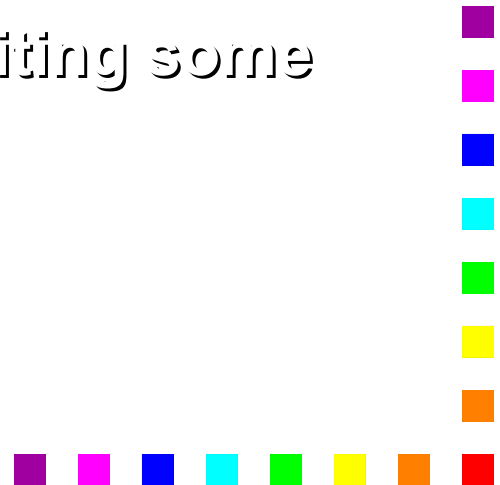


Sign Conditions

ArithExp IS [NOT] $\left\{ \begin{array}{l} \underline{\text{POSITIVE}} \\ \underline{\text{NEGATIVE}} \\ \underline{\text{ZERO}} \end{array} \right\}$

The sign condition determines whether or not the value of an arithmetic expression is less than, greater than or equal to zero.

Sign conditions are just another way of writing some of the Relational conditions.



Complex conditions.

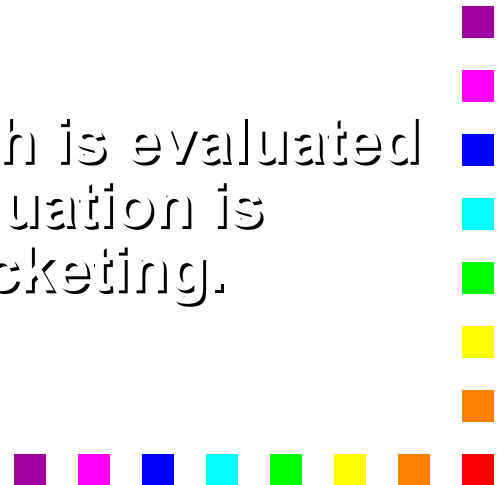
$$\text{Condition} \left\{ \left\{ \begin{array}{c} \text{AND} \\ \text{OR} \end{array} \right\} \text{Condition} \right\} \dots$$

Programs often require conditions which are more complex than single value testing or determining a data class.

Like all other programming languages COBOL allows simple conditions to be combined using **OR** and **AND** to form composite conditions.

Like other conditions, a complex condition evaluates to true or false.

A complex condition is an expression which is evaluated from left to right unless the order of evaluation is changed by the precedence rules or bracketing.



Complex conditions have precedence rules too.

Precedence Rules.

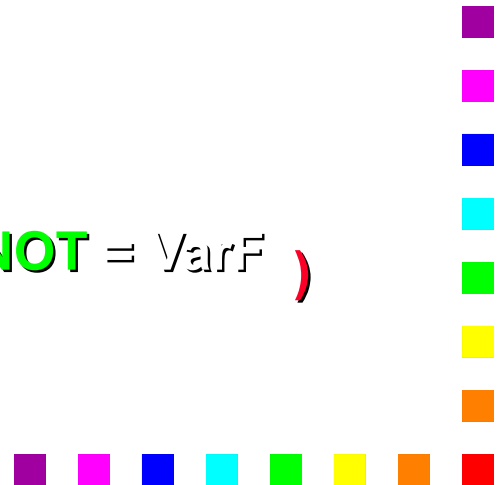
1. **NOT** = **
2. **AND** = * or /
3. **OR** = + or -

Just like arithmetic expressions, complex conditions are evaluated using precedence rules and the order of evaluation may be changed by bracketing.

Examples

```
IF ( Row > 0 ) AND ( Row < 26 ) THEN  
    DISPLAY "On Screen"  
END-IF
```

```
IF ( VarA > VarC ) OR ( VarC = VarD ) OR ( VarA NOT = VarF )  
    DISPLAY "Done"  
END-IF
```



Implied Subjects.

When a data item is involved in a relation condition with each of a number of other items it can be tedious to have to repeat the data item for each condition. For example,

OR AND
AND OR
AND AND

In these situations COBOL provides an abbreviation mechanism called **implied subjects**.

The statements above may be re-written using implied subjects as;

OR AND
AND OR
AND AND

Implied Subjects
TotalAmt
Grade =
VarA >

Nested IFs

```
IF ( VarA < 10 ) AND ( VarB NOT > VarC ) THEN
    IF VarG = 14 THEN
        DISPLAY "First"
    ELSE
        DISPLAY "Second"
    END-IF
ELSE
    DISPLAY "Third"
END-IF
```

VarA	VarB	VarC	VarG	DISPLAY
3 3 3 13	T T T F	4 4 3 15	14 15 14 14	First Second Third Third

Condition Names.

IF VarA GREATER THAN VarB THEN Action

**Condition is either
TRUE or False**

Wherever a **condition** can occur, such as in an IF statement or an EVALUATE or a PERFORM..UNTIL, a **CONDITION NAME** (Level 88) may be used.

A Condition Name is essentially a BOOLEAN variable which is either **TRUE** or **FALSE**.

Example.

IF StudentRecord = HIGH-VALUES THEN *Action*

IF EndOfStudentFile THEN *Action*



Defining Condition Names.

88 ConditionName $\left\{ \begin{array}{l} \underline{\text{VALUE}} \\ \underline{\text{VALUES}} \end{array} \right\} \left\{ \begin{array}{l} \text{Literal} \\ \text{LowValue } \left\{ \begin{array}{l} \underline{\text{THROUGH}} \\ \underline{\text{THRU}} \end{array} \right\} \text{HighValue} \end{array} \right\} \dots$

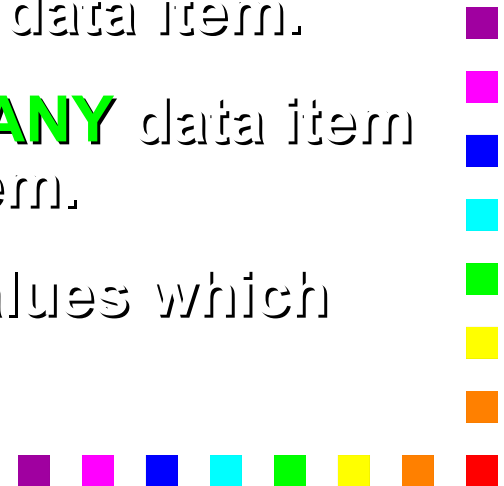
Condition Names are defined in the DATA DIVISION using the special level number **88**.

They are always associated with a data item and are defined immediately after the definition of the data item.

A condition name takes the value **TRUE** or **FALSE** depending on the value in its associated data item.

A Condition Name may be associated with **ANY** data item whether it is a group or an elementary item.

The VALUE clause is used to **identify** the values which make the Condition Name TRUE.



01	CityCode	PIC 9	VALUE 5.
88	Dublin	VALUE	1.
88	Limerick	VALUE	2.
88	Cork	VALUE	3.
88	Galway	VALUE	4.
88	Sligo	VALUE	5.
88	Waterford	VALUE	6.
88	UniversityCity	VALUE	1 THRU 4.

```
IF Limerick
  DISPLAY "Hey, we're home."
END-IF
IF UniversityCity
  PERFORM CalcRentSurcharge
END-IF
```

City Code

5

Dublin	FALSE
Limerick	FALSE
Cork	FALSE
Galway	FALSE
Sligo	TRUE
Waterford	FALSE
UniversityCity	FALSE

01	CityCode	PIC 9	VALUE 5.
88	Dublin	VALUE	1.
88	Limerick	VALUE	2.
88	Cork	VALUE	3.
88	Galway	VALUE	4.
88	Sligo	VALUE	5.
88	Waterford	VALUE	6.
88	UniversityCity	VALUE	1 THRU 4.

```
IF Limerick
  DISPLAY "Hey, we're home."
END-IF
IF UniversityCity
  PERFORM CalcRentSurcharge
END-IF
```

City Code

2

Dublin	FALSE
Limerick	TRUE
Cork	FALSE
Galway	FALSE
Sligo	FALSE
Waterford	FALSE
UniversityCity	TRUE

01	CityCode	PIC 9	VALUE 5.
88	Dublin	VALUE	1.
88	Limerick	VALUE	2.
88	Cork	VALUE	3.
88	Galway	VALUE	4.
88	Sligo	VALUE	5.
88	Waterford	VALUE	6.
88	UniversityCity	VALUE	1 THRU 4.

```
IF Limerick
  DISPLAY "Hey, we're home."
END-IF
IF UniversityCity
  PERFORM CalcRentSurcharge
END-IF
```

City Code

6

Dublin	FALSE
Limerick	FALSE
Cork	FALSE
Galway	FALSE
Sligo	FALSE
Waterford	TRUE
UniversityCity	FALSE

```
01 InputChar    PIC X.  
   88 Vowel     VALUE "A", "E", "I", "O", "U".  
   88 Consonant  VALUE "B" THRU "D", "F", "G", "H"  
                        "J" THRU "N", "P" THRU "T"  
                        "V" THRU "Z".  
   88 Digit      VALUE "0" THRU "9".  
   88 LowerCase  VALUE "a" THRU "z".  
   88 ValidChar  VALUE "A" THRU "Z", "0" THRU "9".
```

```
IF ValidChar  
    DISPLAY "Input OK."  
END-IF  
IF LowerCase  
    DISPLAY "Not Upper Case"  
END-IF  
IF Vowel  
    Display "Vowel entered."  
END-IF
```

Input Char

E

Vowel **TRUE**
Consonant **FALSE**
Digit **FALSE**
LowerCase **FALSE**
ValidChar **TRUE**

```
01 InputChar    PIC X.  
   88 Vowel     VALUE "A", "E", "I", "O", "U".  
   88 Consonant  VALUE "B" THRU "D", "F", "G", "H"  
                        "J" THRU "N", "P" THRU "T"  
                        "V" THRU "Z".  
   88 Digit      VALUE "0" THRU "9".  
   88 LowerCase  VALUE "a" THRU "z".  
   88 ValidChar  VALUE "A" THRU "Z", "0" THRU "9".
```

```
IF ValidChar  
    DISPLAY "Input OK."  
END-IF  
IF LowerCase  
    DISPLAY "Not Upper Case"  
END-IF  
IF Vowel  
    Display "Vowel entered."  
END-IF
```

Input Char

4

Vowel **FALSE**
Consonant **FALSE**
Digit **TRUE**
LowerCase **FALSE**
ValidChar **TRUE**


```
01 InputChar    PIC X.  
   88 Vowel     VALUE "A", "E", "I", "O", "U".  
   88 Consonant  VALUE "B" THRU "D", "F", "G", "H"  
                        "J" THRU "N", "P" THRU "T"  
                        "V" THRU "Z".  
   88 Digit      VALUE "0" THRU "9".  
   88 LowerCase  VALUE "a" THRU "z".  
   88 ValidChar  VALUE "A" THRU "Z", "0" THRU "9".
```

```
IF ValidChar  
    DISPLAY "Input OK."  
END-IF  
IF LowerCase  
    DISPLAY "Not Upper Case"  
END-IF  
IF Vowel  
    Display "Vowel entered."  
END-IF
```

Input Char

g

Vowel **FALSE**
Consonant **FALSE**
Digit **FALSE**
LowerCase **TRUE**
ValidChar **FALSE**

```
01 EndOfFileFlag    PIC 9 VALUE 0.  
88 EndOfFile        VALUE 1.
```

EndOfFileFlag

0

EndOfFile

```
READ InFile  
    AT END MOVE 1 TO EndOfFileFlag  
END-READ  
PERFORM UNTIL EndOfFile  
    Statements  
    READ InFile  
        AT END MOVE 1 TO EndOfFileFlag  
    END-READ  
END-PERFORM
```

```
01 EndOfFileFlag    PIC 9 VALUE 0.  
88 EndOfFile        VALUE 1.
```

EndOfFileFlag

1

EndOfFile

```
READ InFile  
    AT END MOVE 1 TO EndOfFileFlag  
END-READ  
PERFORM UNTIL EndOfFile  
    Statements  
    READ InFile  
        AT END MOVE 1 TO EndOfFileFlag  
    END-READ  
END-PERFORM
```

Using the SET verb.

```
01 FILLER          PIC 9 VALUE 0.  
   88 EndOfFile    VALUE 1.  
   88 NotEndOfFile VALUE 0.
```

FILLER

0

EndOfFile **1**
NotEndOfFile **0**

```
READ InFile  
    AT END SET EndOfFile TO TRUE  
END-READ  
PERFORM UNTIL EndOfFile  
    Statements  
    READ InFile  
        AT END SET EndOfFile TO TRUE  
    END-READ  
END-PERFORM  
Set NotEndOfFile TO TRUE.
```

Using the SET verb.

```
01 FILLER          PIC 9 VALUE 0.  
   88 EndOfFile    VALUE 1.  
   88 NotEndOfFile VALUE 0.
```

FILLER

1

EndOfFile 1
NotEndOfFile 0

```
READ InFile  
    AT END SET EndOfFile TO TRUE  
END-READ  
PERFORM UNTIL EndOfFile  
    Statements  
    READ InFile  
        AT END SET EndOfFile TO TRUE  
    END-READ  
END-PERFORM  
Set NotEndOfFile TO TRUE.
```

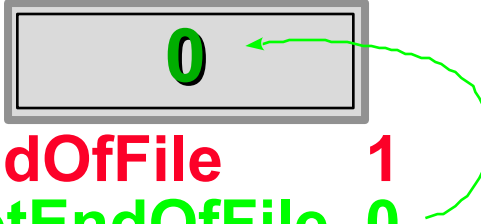
Using the SET verb.

```
01 FILLER          PIC 9 VALUE 0.  
   88 EndOfFile    VALUE 1.  
   88 NotEndOfFile VALUE 0.
```

FILLER

0

EndOfFile 1
NotEndOfFile 0

A diagram consisting of a rectangular box with a black border. Inside the box, the number '0' is displayed in green. A green arrow originates from the right side of the box and points to the value '1' next to 'EndOfFile'. Another green arrow originates from the right side of the box and points to the value '0' next to 'NotEndOfFile'.

```
READ InFile  
    AT END SET EndOfFile TO TRUE  
END-READ  
PERFORM UNTIL EndOfFile  
    Statements  
    READ InFile  
        AT END SET EndOfFile TO TRUE  
    END-READ  
END-PERFORM  
Set NotEndOfFile TO TRUE.
```

```

FD StudentFile.
01 StudentDetails.
    88 EndOfFile    VALUE HIGH-VALUES.
    02 StudentId    PIC 9(7) .
    02 StudentName.
        03 Surname    PIC X(8) .
        03 Initials    PIC XX.
    02 DateOfBirth.
        03 YOBirth    PIC 9(2) .
        03 MOBirth    PIC 9(2) .
        03 DOBirth    PIC 9(2) .
    02 CourseCode    PIC X(4) .
    02 Grant          PIC 9(4) .
    02 Gender          PIC X.

```

PROCEDURE DIVISION.

Begin.

OPEN INPUT StudentFile

READ StudentFile

AT END SET EndOfFile TO TRUE

END-READ

PERFORM UNTIL EndOfFile

DISPLAY StudentId SPACE StudentName SPACE CourseCode

READ StudentFile

AT END SET EndOfFile TO TRUE

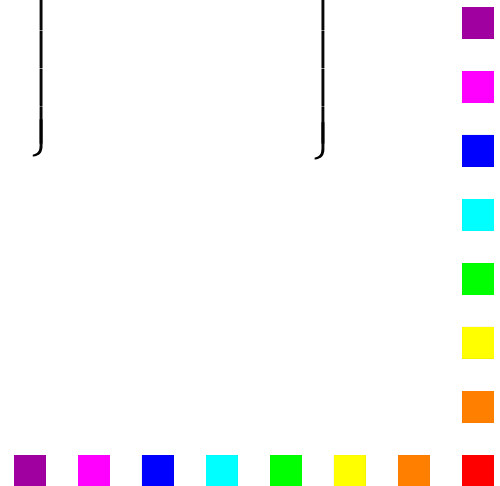
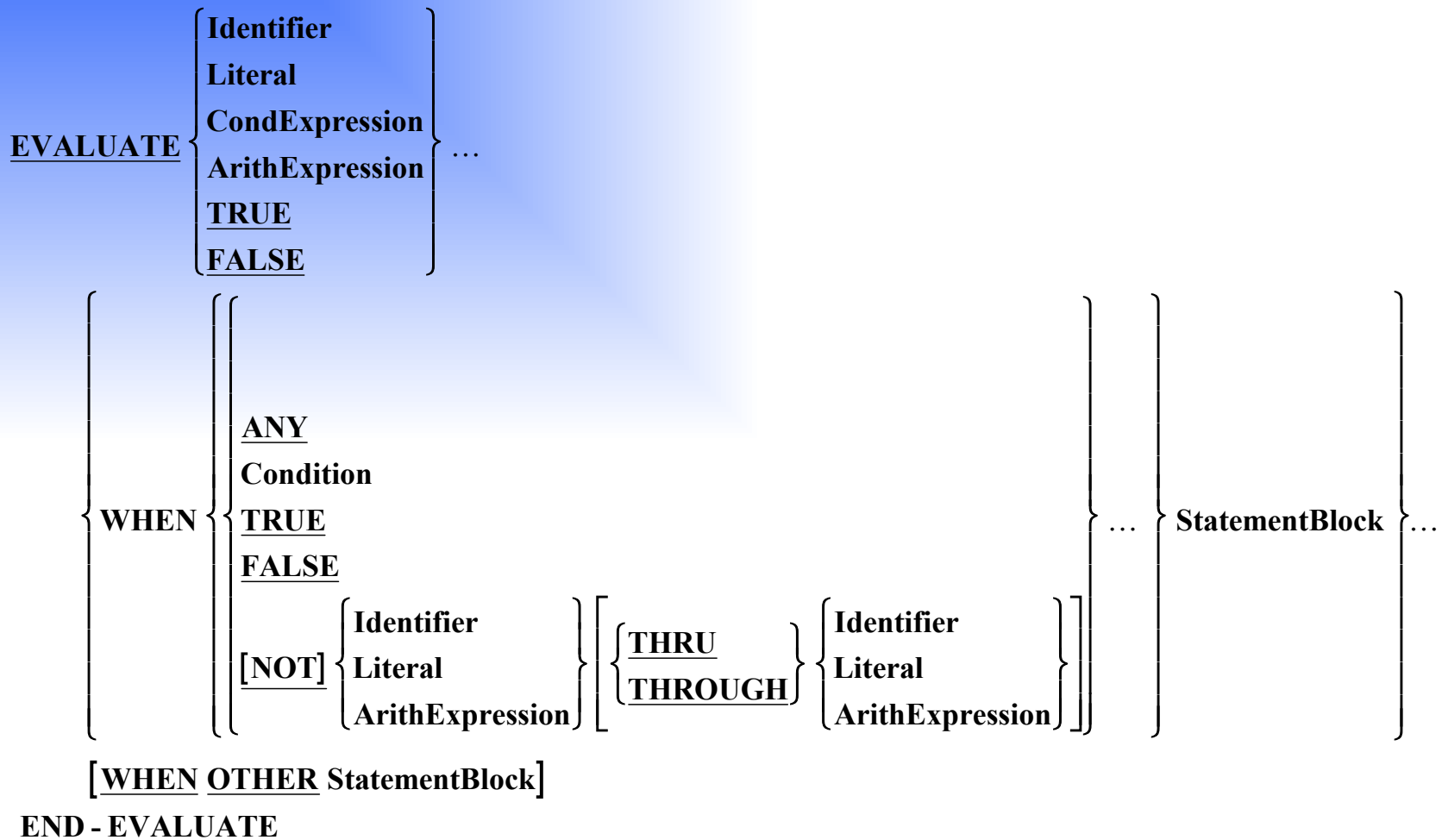
END-READ

END-PERFORM

CLOSE StudentFile

STOP RUN.

The Evaluate



W	I	L	L	I	A	M	S		
---	---	---	---	---	---	---	---	--	--

```

EVALUATE  TRUE      Position
  WHEN L-Arrow      2 THRU 10 PERFORM MoveLeft
  WHEN R-Arrow      1 THRU  9 PERFORM MoveRight
  WHEN L-Arrow           1     MOVE 10 TO Position
  WHEN R-Arrow          10     MOVE  1 TO Position
  WHEN DeleteKey       1     PERFORM CantDelete
  WHEN Character      ANY     PERFORM InsertChar
  WHEN OTHER PERFORM DisplayErrorMessage
END-EVALUATE

```

Decision Table Implementation.

Gender	M	F	M	F	M	F	M	F	
Age	<20	<20	20-40	20-40	40>	40>	20-40	20-40	etc
Service	Any	Any	<10	<10	<10	<10	10-20	10-20	etc
% Bonus	5	10	12	13	20	15	14	23	

```

EVALUATE Gender      TRUE      TRUE
  WHEN  "M"  Age<20      ANY      MOVE 5 TO Bonus
  WHEN  "F"  Age<20      ANY      MOVE 10 TO Bonus
  WHEN  "M"  Age>19 AND <41 Service<10 MOVE 12 TO Bonus
  WHEN  "F"  Age>19 AND <41 Service<10 MOVE 13 TO Bonus
  WHEN  "M"  Age>40      Service<10 MOVE 20 TO Bonus
  WHEN  "F"  Age>40      Service<10 MOVE 15 TO Bonus
      :      :
      :      :
  WHEN  "F"      ANY      Service>20 MOVE 25 TO Bonus
END-EVALUATE
  
```